

Latencia y Optimización de Rendimiento LLM

Módulo 9 · Infraestructura y Despliegue

TTFT vs generation latency, prefill vs decode, prefix caching, speculative decoding y diagnostico de cuellos de botella.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Latencia y Optimización de Rendimiento LLM

La latencia de un sistema LLM tiene dos componentes: TTFT (Time To First Token, el tiempo hasta que el usuario ve el primer carácter de la respuesta) y generation latency (el tiempo total hasta que la respuesta está completa). TTFT es el indicador más crítico para la experiencia de usuario: un TTFT <1 segundo hace que el sistema se sienta responsivo incluso si la respuesta completa tarda 10 segundos. La generation latency completa determina el coste total de la operación.

En 2025, los objetivos típicos de producción para sistemas LLM conversacionales son: TTFT P50 <500ms, TTFT P99 <2s, latencia end-to-end P99 <5s para respuestas de longitud media. Para sistemas de batch processing, el objetivo es máximo throughput (tokens por segundo) en lugar de baja latencia individual. Entender cómo optimizar cada uno requiere identificar el cuello de botella en el pipeline: ¿es el prefill (procesamiento del prompt)? ¿el decode (generación de tokens)? ¿la recuperación RAG? ¿la red?

Los cuellos de botella de latencia en sistemas LLM

Prefill vs decode: dos fases con perfiles distintos

El prefill procesa el prompt completo de entrada para generar los KV caches necesarios para la generación. Es intensivo en cómputo (compute-bound), lineal con la longitud del prompt: un prompt de 10.000 tokens tarda 10x más que uno de 1.000 tokens. El decode genera tokens uno a uno usando el KV cache generado en el prefill. Es intensivo en memoria bandwidth (memory-bandwidth-bound): cada nuevo token requiere leer todo el KV cache de los tokens anteriores. La latencia de decode es aproximadamente lineal con el número de tokens generados y con el tamaño del modelo.

KV cache: el cuello de botella de memoria

El KV (Key-Value) cache almacena los tensores de atención para todos los tokens del contexto. Crece linealmente con la longitud del contexto y con el tamaño del modelo: para Llama 3 70B con un contexto de 8.192 tokens, el KV cache ocupa ~16GB. Con múltiples usuarios concurrentes, el KV cache de todos los usuarios en proceso puede superar la VRAM disponible. vLLM resuelve esto con PagedAttention (gestión paginada del KV cache) que reduce el overhead por fragmentación de memoria y permite mayor concurrencia en la misma VRAM.

Prefix caching: reutilizar el KV cache de prompts comunes

Cuando múltiples peticiones comparten el mismo prefijo (por ejemplo, el mismo system prompt largo de 3.000 tokens), vLLM y TGI pueden cachear el KV cache de ese prefijo y reutilizarlo sin recalcularlo. Esto reduce el TTFT en un 50-90% para peticiones con prefijos compartidos. Es especialmente efectivo para sistemas RAG donde el system prompt incluye documentos recuperados que son similares entre peticiones, o para multi-turn conversations donde los turnos anteriores forman un prefijo común.

Técnicas de optimización de latencia

Streaming para reducir la latencia percibida

Streaming (stream: true) devuelve los tokens al cliente a medida que se generan sin esperar a que la respuesta esté completa. El usuario ve el primer token en el TTFT (tiempo de prefill + tiempo de generación del primer token), que es mucho menor que la latencia end-to-end total. Para una respuesta de 500 tokens que tarda 8 segundos en total, sin streaming el usuario espera 8 segundos; con streaming, el usuario ve los primeros tokens en <1 segundo. Streaming no reduce el coste ni el TTFT, pero mejora dramáticamente la percepción de velocidad.

Speculative decoding: acelerar la generación

Speculative decoding usa un modelo draft pequeño para generar un borrador de K tokens, que el modelo principal verifica en un solo forward pass. Si el modelo principal acepta el borrador (alta probabilidad cuando el draft es bueno), se ahorra el tiempo de K forward passes del modelo principal. Para modelos grandes con baja concurrencia donde el decode es el cuello de botella, speculative decoding puede reducir la latencia de decode 2-3x. vLLM soporta speculative decoding con --speculative-model.

Reducción de la longitud del contexto

La optimización de latencia más directa y sin coste de infraestructura es reducir la longitud del contexto enviado al modelo. Estrategias: truncar el system prompt a lo mínimo necesario, usar reranking para seleccionar solo los K documentos RAG más relevantes (K=3 en lugar de K=10), comprimir el historial de conversación con resúmenes, y usar prompts más concisos. Reducir el contexto de 8.000 a 4.000 tokens puede reducir el TTFT un 40-50% y el coste de inferencia un 30-40%.

SECCIÓN 4 · EJEMPLOS REALES

- Optimización del TTFT en sistema RAG de soporte técnico: el TTFT inicial era de 3.2s (P50) porque el sistema enviaba 10 documentos de contexto RAG (~8.000 tokens) al modelo. Optimizaciones aplicadas: reranking para seleccionar los 3 documentos más relevantes (~2.400 tokens), prefix caching del system prompt fijo (reduce el prefill del system prompt a cero en la mayoría de peticiones). Resultado: TTFT P50 reducido a 0.9s sin cambiar el modelo ni la infraestructura.
- Speculative decoding para asistente de código (latencia crítica): el asistente genera snippets de código de 50-200 tokens con baja concurrencia (max 10 usuarios simultáneos). Codellama 7B como draft model + Codellama 34B como target model con speculative decoding. Reducción de latencia de decode del 40% manteniendo la misma calidad de código. Los tokens aceptados del draft son el 75% en promedio para código Python estándar.
- Prefix caching del corpus RAG en sistema de análisis financiero (Morgan Stanley): los documentos de research financiero recuperados para preguntas del mismo analista en la misma sesión frecuentemente comparten el mismo contexto. TGI v3 con prefix caching: las peticiones sucesivas con el mismo corpus recuperado tienen TTFT 13x menor. Para

sesiones de trabajo largas con el mismo expediente de empresa, el primer request tarda 2.5s; los siguientes <200ms.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Optimizar la latencia antes de medir el cuello de botella. Implementar speculative decoding para un sistema donde el cuello de botella es el prefill (no el decode) no mejorará la latencia. Siempre mide primero: ¿cuánto tiempo es prefill vs decode vs retrieval vs red? Optimiza el mayor contribuidor.

Error 2: Confundir latencia de token con latencia percibida. El TTFT es el indicador de experiencia de usuario más importante en sistemas conversacionales, no la latencia total. Un sistema con streaming que entrega el primer token en 300ms y completa la respuesta en 10 segundos se percibe como más rápido que uno sin streaming que devuelve la respuesta completa en 5 segundos.

Error 3: No considerar el impacto del número de GPUs en la latencia. Tensor parallelism (dividir el modelo entre N GPUs) reduce el tiempo de inferencia pero añade latencia de comunicación entre GPUs. Para modelos que caben en una GPU, usar tensor parallelism de 2 GPUs puede reducir la latencia en un 40% pero no en un 50% por el overhead de comunicación. Para modelos que no caben en una GPU, tensor parallelism es necesario independientemente.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de diagnóstico de latencia para sistemas LLM en producción: instrumenta el pipeline con OpenTelemetry para medir el tiempo de cada componente (retrieval, embed query, construction del prompt, LLM call, post-processing). Identifica el componente que consume mayor tiempo. Si es el LLM call: mide el split prefill/decode. Si prefill domina: reduce el contexto o activa prefix caching. Si decode domina: considera speculative decoding o un modelo más pequeño. Si retrieval domina: optimiza el índice vectorial o el reranker.

Para establecer los SLOs correctos de latencia: mide la latencia actual en producción (TTFT P50, P95, P99 y latencia end-to-end). Define los objetivos basados en los requisitos de UX: conversaciones interactivas requieren TTFT P99 < 2s, batch processing puede tolerar latencias de minutos. Implementa alertas cuando los SLOs se violan con un margen de seguridad (alerta al 80% del umbral del SLO para dar tiempo a reaccionar antes de violarlo).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T6 (vLLM y TGI): PagedAttention y continuous batching son los mecanismos que optimizan la latencia a nivel del servidor de inferencia.
- M9-T5 (Escalabilidad): el warm pool es la herramienta de escalabilidad que más impacta en la latencia de cold start.
- M8-T7 (Observabilidad): el tracing distribuido es la herramienta para diagnosticar cuellos de botella de latencia.

SECCIÓN 8 · RESUMEN EJECUTIVO

La latencia LLM tiene dos componentes: TTFT (tiempo hasta el primer token, experiencia de usuario) y generation latency total (throughput). Los cuellos de botella principales son: prefill (procesamiento del prompt, compute-bound, lineal con longitud del contexto), decode (generación de tokens, memory-bandwidth-bound), KV cache (aumenta con la longitud del contexto). Las optimizaciones más efectivas: streaming (mejora percepción sin cambiar latencia real), reducir el contexto (40-50% de TTFT con 50% menos tokens), prefix caching (TTFT 50-90% menor para prompts compartidos), speculative decoding (decode 40% más rápido). Mide siempre antes de optimizar: identifica el cuello de botella con OpenTelemetry.